Lab: SSL Stripping Attack — Safe Demonstration

Objective: Demonstrate how an attacker positioned between a client and server can downgrade an HTTP→HTTPS connection and observe/modify traffic in a controlled environment.

Lab setup

- **Attacker:** Kali Linux VM
- **Victim:** Windows/Linux VM
- **Target:** Your own local test web application
- Put all VMs on the same **Host-only/Internal Network**
- Do **not** use your college/public Wi-Fi for the experiment.

Execution steps

1. Check the network

On Kali:

```bash
ip addr
```

On the victim:

```cmd
ipconfig
```

Confirm that both machines are on the same isolated lab network.

2. Start a test HTTP/HTTPS application

Use a deliberately vulnerable local application such as OWASP Juice Shop or another application specifically configured for the lab.

Access it from the victim:

```
http://<LAB-SERVER-IP>
```

Verify that the normal HTTP/HTTPS behavior works before introducing the attack.

3. Enable IP forwarding on Kali

```Bash
sudo sysctl -w net.ipv4.ip_forward=1
```

Verify:

```Bash
cat /proc/sys/net/ipv4/ip_forward
```

It should show:

```
1
```

4. Demonstrate ARP interception

For a controlled lab, use an interception tool such as **Ettercap** or **Bettercap** to place Kali between the victim and your test server.

Before running it, identify:

```Bash
ip route
```

and the victim/server IP addresses.

5. Observe the traffic

Use Wireshark on the Kali VM:

Bash

```
sudo wireshark
```

Select the lab network interface and observe the traffic generated by the victim. For HTTP traffic, you can demonstrate that request data is readable.

6. Demonstrate the downgrade concept

The important teaching point is:

```
Normal:

Victim  ──────  HTTPS  ──────>  Server
           encrypted


SSL stripping:

Victim ──  HTTP  ──>  Attacker  ──  HTTPS  ──>  Server
           ↑
        plaintext
```

The attacker maintains HTTPS toward the real server while presenting HTTP to the victim. This is the fundamental SSL-stripping concept.

7. Demonstrate modification safely

Instead of modifying credentials or real websites, create a simple test page such as:

HTML

```
<h1>Original Message</h1>
<p>This is the authorized SSL-stripping lab.</p>
```

Then demonstrate changing only harmless content such as:

```
Original Message
        ↓
Modified Lab Message
```

This lets students observe the integrity problem without capturing real credentials or attacking an external service.

8. Observe with Wireshark

Useful filters include:

```
http
```

and:

```
tls
```

Students should compare:

Traffic	What students observe
HTTP	Content can be read/modified
HTTPS	Application content is encrypted
SSL stripping	Victim can be downgraded to HTTP if the application/configuration permits it

9. Show the protection

Explain that modern HTTPS protections reduce the effectiveness of classic SSL stripping, particularly:

- **HSTS**
- **HSTS preload**
- Correct HTTP→HTTPS redirection
- Secure cookie attributes
- TLS configuration
- Avoiding mixed HTTP/HTTPS content

Student conclusion

> In this experiment, an SSL-stripping attack was demonstrated in an isolated lab environment. The experiment showed that when a victim is downgraded from HTTPS to HTTP, application data can become exposed to an attacker positioned between the victim and server. HTTPS with HSTS and proper secure configuration helps prevent this downgrade.

If you want this specifically as a **Kali Linux practical with exact Bettercap/Ettercap commands, VM IP addresses, Wireshark filters, and expected outputs**, I can format it as a **student-ready experiment sheet**.

Classroom demonstration setup

Use:

- **Kali Linux VM** → attacker
- **Windows VM** → victim/student machine
- **Local test web server** → target
- Network: **Host-only/Internal Network**

Do **not** use Google, banking sites, college Wi-Fi, or real student credentials.

What you show students

Draw this on the board:

```
NORMAL HTTPS

Victim  ————————— HTTPS ————————>  Web Server
              🔒 encrypted


SSL STRIPPING

Victim  —— HTTP —>  Kali  —— HTTPS —> Web Server
          🔓             |
                    attacker
```

Tell students:

> "The victim thinks they are communicating normally, but the attacker keeps the victim side on HTTP while maintaining HTTPS with the real server."

Actual classroom flow

Step 1 — Show the normal website

On the Windows/Victim VM, open your **local test website**:

```
http://<your-test-server-IP>
```

Show students the page.

For example:

```
Welcome to Cyber Security Lab
Username: _______
Password: _______
```

Use **fake credentials only**, such as:

```
student
demo123
```

Step 2 — Show the traffic without the attack

Open Wireshark on Kali and start capturing the lab interface. The

victim submits:

```
Username: student
Password: demo123
```

Because this is deliberately HTTP, demonstrate that the HTTP request can be inspected in Wireshark.

Explain:

> "HTTP does not provide confidentiality. Someone who can observe this traffic may be able to read the application data."

Step 3 — Introduce SSL stripping

Now explain that the attacker attempts to position themselves between the victim and server:

```
Before:

Victim ————————————> Server


After:
```

```
Victim ──> Kali ──> Server
           ↑
        attacker
```

For the classroom, use **Bettercap/Ettercap only against your isolated lab VMs**.

Step 4 — Demonstrate the important part

Have the victim access the **local test application** again. The

intended demonstration is:

```
Victim
   |
   | HTTP
   ↓
Kali
   |
   | HTTPS
   ↓
Test Server
```

Now show students that the victim-side traffic is no longer protected by HTTPS.

The easiest way to explain the attack

Use this example:

Without SSL stripping

```
Victim → HTTPS → Server

"Password = demo123"
        🔒
```

With SSL stripping

```
Victim → HTTP → Kali → HTTPS → Server

"Password = demo123"
        ↑
   readable here
```

So the key lesson is **not simply "Kali can hack HTTPS."**

The lesson is:

> **SSL stripping attempts to prevent the victim from establishing HTTPS, while the attacker communicates with the server using HTTPS.**

Then show the defense

After the demonstration, explain:

```
HSTS
 ↓
Browser remembers:
"Use HTTPS only"
 ↓
SSL downgrade becomes much harder/prevented
```

Also show students the difference:

Situation	Data protection
HTTP	✗ Not encrypted
HTTPS	✓ Encrypted
SSL stripping	⚠ Attempts to downgrade victim to HTTP
HTTPS + HSTS	✓ Helps prevent downgrade